

The Shakedown

A Companion to the Entmoot Architecture Paper

April 2026

Abstract

Between 17 and 27 April 2026 we shipped a dense sequence of patch releases across two layered systems: the Pilot fork moved from `v1.7.2-jf.1` to `v1.9.0-jf.15.21`, and Entmoot moved from `v1.0.0` to `v1.5.25`, followed by the current member-profile and group-display-metadata work. Every patch was driven by a symptom observed on a three-machine live demo across three continents: a laptop on a UAE carrier-grade NAT, a Raspberry Pi in Italy on Fastweb residential, and a Hetzner VPS in Germany. The CHANGELOGs document what changed, but their language assumes a reader who already speaks networking. This document is the narrative companion: it explains what the words mean, what actually went wrong on the wire, and why each fix is the canonical one. It is meant for anyone who wants to understand the project without first reading RFCs.

Contents

1	What this document is	2
2	The world before the shakedown	2
2.1	Three machines, three layers	2
2.2	Five metaphors that are load-bearing	3
2.3	The four verbs you will see	3
3	Glossary	4
4	The Pilot fixes, in order	7
4.1	<code>v1.7.2-jf.1</code> : don't punch at private addresses	7
4.2	<code>v1.7.2-jf.2</code> : keep the tunnel alive across rekeying	7
4.3	<code>v1.7.2-jf.3</code> : independent beacon keepalive, and persistent visibility	8
4.4	<code>v1.8.0-jf.1</code> : pluggable transports and TCP fallback	8
4.5	<code>v1.9.0-jf.1</code> : gossip-based peer discovery	9
4.6	<code>v1.9.0-jf.2</code> : NAT-drift refresh and macOS code-sign workaround	9
4.7	<code>v1.9.0-jf.3</code> : automatic recovery from one-sided restart	10
4.8	<code>v1.9.0-jf.4</code> : reply on the path the request came in on	10
4.9	<code>v1.9.0-jf.5</code> : clean up all relay state	11
4.10	<code>v1.9.0-jf.6</code> : don't dial yourself	11
5	The Entmoot fixes, in order	12
5.1	<code>v1.0.0</code> : the foundations	12
5.2	<code>v1.0.2</code> : retries and anti-entropy	12
5.3	<code>v1.0.3</code> : publish returns immediately	13
5.4	<code>v1.0.4</code> : Plumtree	13
5.5	<code>v1.0.5</code> : re-fanout on every path	14

5.6	v1.0.6: trust-aware fanout, parallel sends, dial-backoff	14
5.7	v1.0.7: inline bodies and jittered backoff	15
5.8	v1.0.8: invites should not include the issuer	15
5.9	v1.1.0: one persistent session per peer	16
5.10	v1.2.0: transport-endpoint advertisements over Entmoot gossip	17
5.11	v1.2.1: Range-Based Set Reconciliation, background anti-entropy, and tunnel-up trigger	17
6	The second shakedown: hide-IP, TURN, and port 1004	18
6.1	Pilot jf.8–jf.11b: TURN and push rotation	19
6.2	Pilot jf.12–jf.15.7: rendezvous and bilateral TURN	19
6.3	Entmoot v1.5.1–v1.5.10: stale streams and reconcile recovery	20
6.4	Pilot jf.15.8–jf.15.21: the stream layer becomes explicit	20
6.5	Entmoot v1.5.11–v1.5.19: mobile service peer primitives and release observability	21
6.6	Entmoot v1.5.20–v1.5.25: executable ESP, multi-group runtime, raw Pilot streams, and hostname-aware members	21
7	The big picture: how the layers interacted	22
8	Cheat sheet	23

1 What this document is

The Entmoot research paper (`paper/main.tex`) describes the protocol as it is designed. This companion describes what went wrong in the first field deployment and how each problem was resolved. If the research paper is the blueprint, this is the engineer’s diary kept while wiring the building.

Two things distinguish this document from the CHANGELOGs:

- It assumes you have never heard of NAT traversal, gossip protocols, or Merkle trees. Section 2 sets up the mental model; Section 3 defines every technical term used later.
- It explains each fix in four parts: what the user saw, what was really happening, what we changed, and why that change is the right shape. Each piece of jargon is introduced in context the first time it appears.

If you only read three sections, read Section 2 for the mental model, Section 3 for vocabulary, and Section 7 for the shape of what went wrong.

2 The world before the shakedown

2.1 Three machines, three layers

The deployment under test has three participants:

Laptop A MacBook in the United Arab Emirates, behind a mobile carrier’s network. The carrier’s equipment rewrites every outbound packet so the laptop has no meaningful “address” on the public internet.

Phobos A Raspberry Pi sitting on a residential Italian internet connection. Also behind a home router that rewrites addresses.

VPS A small Linux server rented from Hetzner in Germany. It has a real public IP address that the internet can reach directly.

Each machine runs two programs that work together:

pilot-daemon The lower layer. Its job is to open an encrypted pipe to any other Pilot node, identified by a stable numeric address, regardless of which network the other node sits on. Think of it as the post office that figures out the physical route from sender to recipient.

entmootd join The upper layer. Its job is to participate in a group: receive messages that anyone else in the group publishes, and push its own. Think of it as the language the letters are written in, on top of the postal service underneath.

The line between the two matters. Many of the fixes in this document look like problems in the upper layer but were really problems in the lower layer percolating upward. See Section 7.

2.2 Five metaphors that are load-bearing

These stand in for full technical definitions throughout the document. The precise definitions are in Section 3.

NAT is a hotel switchboard. When you dial out from your hotel room, the switchboard remembers the outbound line and routes any reply back to your room. If the switchboard has no prior record of you calling the outside party, an incoming call cannot reach you. Home routers, mobile carriers, and corporate firewalls all act as switchboards. NAT traversal is the art of getting two switchboards to each open a line at the same moment so voices can flow.

Gossip is rumour spreading. When a node learns something new, it whispers the news to a small random subset of its neighbours. Each of them whispers onward until everyone knows. The mathematics works out: with enough whispering, convergence is fast and robust even if some neighbours are offline.

A Merkle root is an inventory total. Two warehouses can confirm their stock is identical by comparing a single number (the total), rather than listing every box. If the totals match, the inventories are identical; if they differ, the warehouses drill into their records to find the gap. Merkle trees make this check cryptographically strong.

A handshake is a mutual nameplate exchange. Neither side speaks in earnest until both have shown signed identification and exchanged a shared secret. Afterwards their conversation is encrypted with keys only they can derive.

A relay is a forwarding post office. When two hosts cannot reach each other directly, a mutual acquaintance passes letters between them. Slower than direct delivery, but the only option when both sides are behind switchboards that will not cooperate.

2.3 The four verbs you will see

Join. A machine loads an invite (a signed document that names the group and the current roster) and starts participating. Joining involves contacting at least one existing member, downloading missing history, and announcing itself.

Publish. Author a message, sign it, store it locally, hand it to the gossip layer for dissemination.

Tail. Subscribe to live messages in a group.

Query. Read the local store directly for already-received messages, with filters for author, topic, and time.

3 Glossary

Every term bolded on first use. References point at the authoritative source for each concept.

Cryptography and identity

Ed25519 signature.

A fast, deterministic digital signature based on elliptic curves. Used to prove a message was written by a specific key. (RFC 8032.)

X25519 key exchange.

A procedure that lets two parties derive the same secret without anyone listening being able to learn it. (RFC 7748.)

AES-256-GCM.

A symmetric encryption scheme that also detects tampering, used to encrypt messages once X25519 has produced a shared key. (NIST SP 800-38D.)

Signed append-only log.

A list of entries that can only grow. Each entry is signed, so nothing can be silently edited after the fact. Entmoot’s group roster is one of these. (Certificate Transparency, RFC 9162, uses the same primitive.)

Handshake / trust pair.

A one-time mutual authorisation step where two Pilot daemons each approve the other before traffic flows. Without this pair the daemon will reject the other’s packets at the front door.

Rekey.

Replacing session keys periodically, so a leaked key only compromises a short window of past traffic. (NIST SP 800-57.)

Gossip and dissemination

Fanout.

How many peers a publisher pushes a new message to. Each of those peers may fanout again, so a small fanout per hop still reaches everyone quickly.

Plumtree (eager + lazy push).

A gossip design that builds a spanning tree for fast delivery of full message bodies (eager push) and falls back to broadcasting only message IDs on non-tree links (lazy push), to repair gaps cheaply. (Leitão et al., SRDS 2007.)

IHAVE / GRAFT / PRUNE.

The three control messages that make Plumtree’s tree self-healing. IHAVE says “I have message X but am not sending it to you”; GRAFT says “I did not receive X within the timeout, please send it and promote me back onto the eager branch”; PRUNE says “I already had X via another path, stop sending duplicates, demote me off the eager branch”.

First-seen forwarding.

The rule that every node forwards a message exactly the first time it sees it, regardless of which path it arrived on. Duplicates are dropped. (Demers et al., PODC 1987.)

Anti-entropy reconciliation.

A background synchronisation round between two peers that compares inventory (Merkle roots), finds the gap, and exchanges the missing messages. Runs when a peer reconnects after a disconnection, and periodically otherwise. Scuttlebutt, Riak, and Cassandra all use variants.

Retry budget.

The maximum number of times a transient failure will be retried before giving up. Bounded so that one dead peer cannot cause infinite work. (Google SRE Book, “Handling Overload”.)

Decorrelated-jitter backoff.

A retry-delay formula that picks the next wait randomly between a base and three times the previous wait. Unlike fixed exponential doubling, this prevents many nodes from retrying in lockstep after a shared hiccup. (Marc Brooker, AWS 2015.)

Dial-backoff cache.

A per-peer memory of recent failed connection attempts. Further attempts to the same address short-circuit immediately until the window reopens. (go-libp2p `swarm.DialBackoff`.)

MQTT topics.

Slash-separated names for subjects (`chat/eng/alerts`). Subscribers can use `+` to match any one segment and `#` to match the tail. (MQTT v5 OASIS spec.)

Addresses, NAT, and relays**NAT (Network Address Translation).**

A router function that maps many internal addresses to one public one. (RFC 3022.)

RFC 1918 address.

An internal-only IPv4 range that is never routable on the public internet: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. If you see one in a network trace it is a sign that the real address has been obscured.

Carrier-grade NAT (CGN / CGNAT).

A second layer of NAT applied by the ISP, sharing a small pool of public addresses across many subscribers. Common in mobile and recent residential deployments. (RFC 6598.)

STUN.

A short protocol a host can run against a well-known server to learn what public address its NAT currently presents to the outside world. (RFC 8489.)

NAT hole-punching.

Two hosts behind NATs send packets toward each other at the same moment so that both NATs open a mapping on each side, letting subsequent traffic flow through. (RFC 5128, §3.3.)

Symmetric NAT.

A NAT that chooses a new external port per destination, which breaks naive hole-punching because the port you learned via STUN is not the port another peer will dial. (RFC 3489, §5.)

Relay fallback.

When two peers cannot reach each other directly, a third host reachable to both forwards packets between them. (TURN, RFC 8656.)

Registry / rendezvous server.

The well-known server that every Pilot daemon contacts on startup to announce its current address and look up peers. Pilot uses `34.71.57.205`.

Beacon.

A companion to the registry that handles NAT-punch coordination and relays traffic between peers that need one.

Keepalive.

A small periodic packet sent on an otherwise-idle tunnel so the NAT does not forget the mapping. (RFC 1122, §4.2.3.6.)

Spurious RST.

A reset packet that tears down a TCP-like connection without a genuine protocol reason, usually caused by a middlebox or stale NAT state. (RFC 5961.)

Pilot-specific terms**Node ID.**

A 32-bit integer Pilot assigns at registration, bound to the daemon's Ed25519 key. The group uses these as stable addresses.

PILA.

Pilot's authenticated handshake and key-exchange frame. When a peer cannot decrypt a packet, sending an unsolicited PILA asks the other side to re-handshake.

Tunnel.

The encrypted, authenticated channel between two Pilot nodes. Once established, all application traffic flows inside it.

Stream dial.

Opening a new logical stream on top of an existing tunnel (analogous to opening a TCP connection to a specific port on an already-reachable host).

ErrDialToSelf.

A named error returned when a daemon tries to dial its own Node ID. Mirrors the same guard in `go-libp2p-swarm`.

viaRelay.

A per-peer boolean Pilot tracks to remember whether the latest authenticated packet from that peer arrived directly or via the beacon relay.

Stream multiplexing (yamux).

A technique for running many logical streams over one long-lived transport connection, with each stream opening in zero round-trips. Entmoot uses `hashicorp/yamux` to multiplex gossip frames over a single Pilot tunnel per peer, avoiding a per-frame stream dial. The same library is used by Consul, Nomad, Vault, and libp2p's yamux transport.

macOS operations**Ad-hoc code signing.**

Signing a macOS binary with an empty identity (-). The signature is not notarised and carries no chain of trust, but it satisfies the local kernel's check. Go's cross-compiled binaries ship this way by default.

Gatekeeper.

The macOS subsystem that decides whether a binary may run. On Tahoe (26.2) an ad-hoc-signed binary downloaded from the internet triggers a stall at `_dyld_start` until Gatekeeper finishes verifying it (which, for a local daemon invocation, never terminates visibly).

xattr -cr.

Recursively removes the `com.apple.quarantine` extended attribute from a directory, which is what Gatekeeper keys its verification on.

4 The Pilot fixes, in order

Pilot’s patches came in three clusters: `v1.7.2-jf.1` through `v1.7.2-jf.3` tightened the existing transport. `v1.8.0-jf.1` and `v1.9.0-jf.1` introduced two new pieces: a pluggable transport layer with a TCP fallback, and a gossip-based peer-discovery overlay. `v1.9.0-jf.2` through `v1.9.0-jf.6` are a tight cluster of reliability fixes driven by the three-machine demo.

4.1 `v1.7.2-jf.1`: don’t punch at private addresses

What broke. When the laptop and VPS tried to establish a tunnel, a log line appeared on the laptop:

```
STUN returned private/unusable IP, discarding
stun_addr=10.128.0.12
```

Shortly after, the tunnel handshake timed out. Packet captures showed that both daemons were sending NAT-punch packets toward `10.128.0.12`, which is a private address (see RFC 1918) that does not exist on the public internet.

What it actually was. Pilot’s public STUN endpoint sits behind a Google Cloud load balancer that rewrites every client’s source address to a private internal one (`10.128.0.12`). The daemon had a filter that discarded that private address when choosing its own advertised endpoint, but *the same filter was not applied to the target of outgoing NAT-punch packets*. Every daemon was loyally sending punch traffic to `10.128.0.12`, which is nobody’s computer.

How we fixed it. `handlePunchCommand` in `pkg/daemon/tunnel.go` now drops punches whose target address is in an RFC 1918, loopback, link-local, or unspecified range, logging `skipping NAT punch to non-routable target` and falling through to the existing relay path.

Why the fix is right. The condition (“is this a routable address?”) already existed in the registration path. The patch reuses `isPrivateAddr` symmetrically on the punch path. Two filters with the same definition at two ingress points is a textbook consistency fix.

Tag. `v1.7.2-jf.1`, commit `2d4e657`. Files: `pkg/daemon/tunnel.go`.

4.2 `v1.7.2-jf.2`: keep the tunnel alive across rekeying

What broke. Encrypted tunnels dropped every five minutes on the dot. Log lines like `peer marked for relay` followed the drop; a new tunnel came back up about thirty seconds later.

What it actually was. Pilot rekeys each tunnel periodically (part of the `NetworkSync` cycle). During the key swap, the brief window where neither side knows which key to use caused a few application-level ACKs to be dropped. The daemon’s *dead-peer detector* counted those missed ACKs as silence and concluded the peer was unreachable, tearing the tunnel down.

How we fixed it. `TunnelManager` now calls a new `notifyRekey` callback after a successful key swap. The callback clears the “unacknowledged keepalives” counter and refreshes `LastActivity` on every connection routed over that peer, so ACKs dropped during the key swap no longer contribute to the dead-peer tally.

Why the fix is right. Dead-peer detection should measure liveness across steady state, not across intentional reconfigurations. The fix is a straightforward acknowledgement-counter reset at the boundary. No application-visible change; tunnels simply stop flapping.

Tag. v1.7.2-jf.2. Files: `ports.go`, `tunnel.go`, `daemon.go`.

4.3 v1.7.2-jf.3: independent beacon keepalive, and persistent visibility

What broke. Two issues. First, on consumer NATs whose UDP idle timeout is 30 to 60 seconds, the Pilot registry heartbeat (once every 60 seconds) was sometimes too infrequent to keep the mapping to the beacon alive: relay traffic stopped flowing between registry ticks. Second, running `pilotctl set-public` (which tells the registry “yes, other peers may see my address”) did not survive a daemon restart.

What it actually was. The heartbeat loop was doing double-duty and assumed both the registry and the beacon had similar timeout budgets; they do not. Visibility state lived only in memory, so the daemon forgot it after a restart.

How we fixed it. Two additions. A new `beaconKeepaliveLoop` runs on an independent 25-second tick so the beacon mapping refreshes well under the strictest consumer NAT timeout. `set-public` and the `-public` launch flag now persist to `~/.pilot/config.json` via a new `writeConfigKey` helper, and the daemon reads the persisted value at startup.

Why the fix is right. Two different upstream services should have two different heartbeat cadences calibrated to their respective timeouts. Configuration that the operator set should survive a restart. These are standard daemon-design hygiene.

Tag. v1.7.2-jf.3. Files: new loop in `daemon.go`; `cmd/pilotctl/main.go`; also the installer redirected to fetch the fork’s binaries.

4.4 v1.8.0-jf.1: pluggable transports and TCP fallback

What broke. The UAE carrier was observed dropping outbound UDP packets to the VPS’s port. Pilot’s entire wire protocol was UDP. No UDP meant no tunnel, full stop.

What it actually was. The daemon owned a single `*net.UDPConn` as the only way to move bytes. There was no mechanism to attempt an alternative protocol if UDP was blocked.

How we fixed it. Introduced a `Transport` interface in a new `pkg/daemon/transport` package. Added `UDPTransport` (the existing behaviour, refactored) and `TCPTransport` (new: 4-byte big-endian length-prefix framing, connection pooling per remote address). The registry protocol was extended to carry a list of `endpoints` per peer, one per transport. After direct UDP retries exhaust, `DialConnection` attempts a TCP dial bounded by a five-second timeout before falling through to the existing relay path.

Why the fix is right. Treating the byte-movement layer as an interface is the canonical pattern for multi-network systems (libp2p, Tailscale, QUIC clients). The design preserves wire compatibility: daemons running only UDP still exchange byte-identical frames with pre-fork versions.

Tag. v1.8.0-jf.1. Files: new `pkg/daemon/transport/`, refactored `tunnel.go` and `daemon.go`, extended `pkg/registry`.

4.5 v1.9.0-jf.1: gossip-based peer discovery

What broke. The multi-transport endpoint field needed to propagate through the registry for peers to learn about each others' TCP listeners. The upstream public registry does not know about the fork's new field, so the TCP fallback was useless against an unmodified registry.

What it actually was. The reachability half of peer information (what addresses you can reach me at) was coupled to the identity half (this node ID belongs to this key). Upgrading reachability required an upstream cooperation we could not rely on.

How we fixed it. Added `pkg/daemon/gossip`, which exchanges *signed* endpoint advertisements peer-to-peer over existing encrypted tunnels at a 25-second tick. The PILA handshake frame was extended with a trailing capability bitmap, so upgraded daemons advertise gossip capability without breaking the wire format for unmodified upstream daemons (older peers truncate the extra bytes and never see them).

Why the fix is right. Two sources of truth (identity anchored in the registry, reachability refreshed peer-to-peer) is the same split libp2p's identify protocol and Tailscale's coordination server use. Each has the right incentive: the registry is a stable root, peer gossip is a fast-changing directory.

Tag. v1.9.0-jf.1. Files: new `pkg/daemon/gossip/`, `daemon.go` lifecycle wiring, `pkg/registry` snapshot format.

4.6 v1.9.0-jf.2: NAT-drift refresh and macOS code-sign workaround

What broke. Two seemingly unrelated symptoms. On the laptop, outbound stream dials over an otherwise-healthy tunnel timed out with "dial timeout" even though `pilotctl peers` reported the peer as encrypted and authenticated. On the same laptop, interactively-launched `pilotctl info` hung indefinitely at `_dyld_start`.

What it actually was. The laptop's UAE carrier was rotating its external source port between rekeys. Pilot cached the peer's UDP address on key exchange and never refreshed it afterwards, so the VPS kept sending to the stale port. The carrier's NAT silently dropped everything to that old port. Meanwhile, macOS 26.2 Tahoe's Gatekeeper was stalling every interactively-launched ad-hoc-signed Go binary in early dynamic loader initialisation.

How we fixed it. Two changes. The daemon refreshes its cached peer endpoint inside `handleEncrypted` on any successful authenticated decrypt (guarded by a read-lock pre-check so the write path is not churned). The installer now re-signs binaries with the local ad-hoc identity and clears `com.apple.quarantine` extended attributes on macOS, which satisfies Gatekeeper's first-launch gate.

Why the fix is right. The cached endpoint is the daemon's latest evidence of reachability; every authenticated packet is fresh evidence. The endpoint should track the evidence. The macOS workaround is what every cross-compiled Go tool needs under Tahoe; adding it to the installer once beats every operator discovering it the hard way.

Tag. v1.9.0-jf.2. Files: `tunnel.go`, `install.sh`.

4.7 v1.9.0-jf.3: automatic recovery from one-sided restart

What broke. When the VPS daemon restarted, its in-memory session keys were gone, but the laptop still had the old keys and kept sending encrypted traffic. The VPS logged

encrypted packet from node but no key peer_node_id=45491

forever. The two sides never spontaneously re-handshook. A manual daemon restart on the laptop was the only way out.

What it actually was. The receiving side could tell it had no key for a peer, but it had no policy to do anything about it. The sending side saw its own keepalive ACKs succeeding at the tunnel level and had no way to observe the other side’s confusion.

How we fixed it. On any decrypt failure caused by missing crypto state, the daemon emits an unsolicited PILA (handshake initiation) to the source address of the offending frame, rate-limited to once per minute per peer (so a hostile sender cannot amplify the mechanism into a reflection attack). Recovery completes within 200 to 300 milliseconds of the peer’s next keepalive.

Why the fix is right. The fix puts the observation and the remediation on the same side: the node that can detect the desync is the one that asks for a re-handshake. The rate-limit prevents abuse. Equivalent to the “force rekey on decrypt fail” rule WireGuard applies to its roaming handshake.

Tag. v1.9.0-jf.3. Files: `tunnel.go` (`maybeSendRecoveryPILA`).

4.8 v1.9.0-jf.4: reply on the path the request came in on

What broke. Tunnels showed `encrypted=yes auth=yes relay=true` in the peer table, and keepalives flowed both ways, but any attempt to open a new application-layer stream timed out. This affected every VPS-to-laptop dial for the whole day.

What it actually was. Pilot kept two maps: one of direct UDP addresses (`peers`) and a boolean flag (`relayPeers`) saying whether a peer was reachable via relay. The two could disagree across daemons: A’s flag could say “via relay” while B’s map still held a stale direct address for A. Worse, when a relay frame arrived from an unknown peer, the daemon cached the beacon’s address as the peer’s direct address, which then polluted same-LAN detection, the freshly-added NAT-drift refresh, and the direct-UDP fallback path.

How we fixed it. Replaced both maps with a single `paths map[uint32]*peerPath`. Each entry records the *most recent authenticated ingress path*: either a direct UDP address or “via relay”. Every authenticated decrypt updates the entry. Every outbound reply is routed on the same path the last request arrived on.

Also: relay round-trip time is two to three times direct, so the SYN retransmission budget starts at three seconds when the peer is in relay mode (up from one second).

Why the fix is right. This is the “reply on ingress” pattern WireGuard uses for roaming, and what Tailscale calls “DERP sticky”. Symmetry between the two daemons falls out of each one observing its own ingress traffic, instead of requiring both sides to agree on a shared model of the network.

Tag. v1.9.0-jf.4. Files: `tunnel.go`, `tunnel_path_test.go`.

4.9 v1.9.0-jf.5: clean up all relay state

What broke. A VPS restart on v1.9.0-jf.4 left relay-only peers in the “encrypted packet from no key” loop indefinitely. The jf.3 auto-recovery fired, but the unsolicited PILA went out to a zero-address placeholder and was dropped silently.

What it actually was. Several per-peer state structures were not fully cleared across `AddPeer` or `RemovePeer`, so stale values from prior sessions poisoned the new session. Also, `maybeSendRecoveryPILA` unconditionally wrote to the direct UDP address, even when the triggering frame had arrived via relay.

How we fixed it. Five small changes, all about per-peer hygiene:

- Recovery PILA now routes through the beacon when the triggering frame arrived via relay.
- `AddPeer` clears `viaRelay` so a fresh direct endpoint is authoritative.
- `RemovePeer` wipes all per-peer state, not just two of the seven structures.
- `handlePacket` no longer auto-adds a peer under the relay-origin zero-address placeholder.
- `maybeSendRecoveryPILA` checks the UDP socket is non-nil before writing (defensive guard during startup).

Why the fix is right. `RemovePeer` should be “reset all per-peer state”, not “reset some of it”. This is WireGuard’s `wg set peer ...remove` contract. The PILA routing fix matches the same reply-on-ingress principle introduced in jf.4, applied to the recovery path.

Tag. v1.9.0-jf.5. Files: `tunnel.go`, `daemon.go`.

4.10 v1.9.0-jf.6: don’t dial yourself

What broke. Gossip propagation inflated from sub-second to minute-scale latencies. The VPS’s `pilot-daemon` was burning 210% CPU (two cores) and pushing roughly 5,900 packets per second, most of them looping to itself.

What it actually was. Every Entmoot invite minted to that point listed the issuer as a bootstrap peer, including on the issuer’s own machine. Pilot’s `ensureTunnel` (callable from code paths other than the Entmoot bootstrap filter) had no guard against dialling the local Node ID. On a multi-homed host the “same-LAN peer detected” logic then matched both a Docker bridge address (172.17.0.1) and the public IP as valid local endpoints, so the daemon established multiple duplicate self-tunnels, each happily retransmitting into the other.

How we fixed it. `ensureTunnel` and `DialConnection` now short-circuit with a typed sentinel `protocol.ErrDialToSelf` when given the local Node ID. The invite minter was also fixed (see Entmoot v1.0.8).

Why the fix is right. `ErrDialToSelf` is the canonical name for this guard (go-libp2p swarm, for one, uses the same pattern). Returning a typed sentinel rather than silently succeeding makes caller-side invariant violations visible. Defence in depth: the fix lands at both the caller (the invite minter) and the callee (the daemon), so a future invite that happens to include self still cannot cause a storm.

Tag. v1.9.0-jf.6. Files: daemon.go.

5 The Entmoot fixes, in order

Entmoot’s releases are easier to summarise: each one was driven by a symptom on the live demo, and the fixes moved steadily from “make push-only gossip reliable” (v1.0.2, v1.0.3) to “implement the canonical gossip design” (v1.0.4, v1.0.5) to “tune it for real networks” (v1.0.6, v1.0.7, v1.0.8) to “remove per-frame transport overhead” (v1.1.0).

5.1 v1.0.0: the foundations

Not strictly a fix; this is the shape of v1 that the later releases patch. Five points to remember:

- **SQLite store.** One database per group, write-ahead logging, one database file per group’s messages. Replaces the JSONL backend used in pre-v1 development.
- **Control-socket IPC.** A single long-running `entmootd join` process per host owns the Pilot connection and is the only writer to the store. Short-lived CLI invocations (`publish`, `tail`, `info`) reach it through a Unix-domain socket at `~/.entmoot/control.sock`.
- **Five-command CLI.** `join`, `publish`, `tail`, `query`, `info`. `join` is the only blocking one.
- **Invites with TTL.** A new `ValidUntil` field in the invite bundle; default 24 hours. Expired invites are rejected.
- **Installer and skill.** `install.sh` (prebuilt binary fast path, source fall back) and a `SKILL.md` for agent-driven installation.

Tag. v1.0.0, 17 April 2026.

5.2 v1.0.2: retries and anti-entropy

What broke. Push-only gossip from v1.0.0 dropped any message it failed to deliver on the first try. A peer that was briefly offline silently missed messages forever; the only recovery was a full re-join.

What it actually was. The design had no concept of “try again later” and no background synchronisation to paper over gaps that the push layer missed.

How we fixed it. Two new mechanisms:

- An exponential-backoff retry loop. Transient push and fetch failures are queued; a `retryLoop` goroutine drains the pending queue every 500 ms with delays `1s`, `2s`, `4s`, `8s`, `16s`, `30s`, `60s` $\times$ 4. Exhausted slots are logged and left to the next mechanism.
- Anti-entropy reconciliation. When a peer reconnects (either as inbound accept or as a successful outbound dial), the side that detects the reconnection starts a round with it: request the peer’s Merkle root, compare with local, and if they differ, request the list of message IDs since the local latest timestamp and fetch the missing bodies. Rate-limited to one round per peer per minute.

Two new wire types for the range enumeration: `MsgRangeReq` (0x09) and `MsgRangeResp` (0x0A).

Why the fix is right. Every production gossip system has both a fast path (push with retry) and a slow path (anti-entropy). Cassandra, Riak, Scuttlebutt: all pair the two. Push handles the common case; anti-entropy is the safety net.

Tag. v1.0.2, 19 April 2026.

5.3 v1.0.3: publish returns immediately

What broke. `entmootd publish` took 30 seconds to return on every call, and the return always printed an IPC read-response timeout, even when the message had been stored locally and was propagating normally.

What it actually was. The publish handler tried to deliver the message to all peers synchronously before acknowledging the CLI. One slow peer blocked the whole command.

How we fixed it. The handler now returns as soon as the message is durably written to the local store. Peer delivery happens asynchronously through the retry scheduler. Publish latency drops to single-digit milliseconds regardless of peer reachability.

Why the fix is right. This is the industry-standard publish contract. Kafka durable-log commit, NATS fire-and-forget, RabbitMQ publisher confirms: “publish” means “accepted for delivery”, not “delivered to every subscriber”. The retry scheduler and anti-entropy reconciliation from v1.0.2 already carry the delivery guarantee.

Tag. v1.0.3, 19 April 2026.

5.4 v1.0.4: Plumtree

What broke. Push-only gossip struggled under asymmetric reachability. On the canary, the laptop and Phobos had no direct edge: neither could reach the other at the transport layer. A message from Phobos reached the VPS but never reached the laptop, because Entmoot had no re-forwarding step.

What it actually was. The gossip design was “publisher fans out; receivers store and stop”. A partial topology guaranteed gaps that only anti-entropy could eventually repair, at the cost of up to a minute of delay.

How we fixed it. Switched to a Plumtree-style protocol (Leitão, Pereira, Rodrigues, SRDS 2007).

Each peer sits in exactly one of two sets per message: `eagerPushPeers` (get the full body up front) or `lazyPushPeers` (get only the ID). Three new control messages maintain this split:

- **IHAVE** (0x0B): “I have message X, but I am sending only the ID”.
- **GRAFT** (0x0C): “I heard IHAVE for X, did not receive the body within three seconds, please send it and promote me to your eager set”.
- **PRUNE** (0x0D): “I already had X via another path; drop me from your eager set to avoid duplicates”.

Initial state: everyone is eager. Duplicates self-prune into a spanning subset per message. On the canary, a Phobos publish now reaches the laptop via VPS as the re-fanout hop.

Why the fix is right. Plumtree is the canonical way to build a self-healing broadcast tree. Its overhead at steady state is one push per spanning-tree edge, instead of the multiplicative explosion of naive gossip. Riak Core, Helium, Solana, libp2p GossipSub all use variants.

Tag. v1.0.4, 20 April 2026.

5.5 v1.0.5: re-fanout on every path

What broke. After v1.0.4, re-fanout fired only when a message arrived via the live push path. Messages acquired via retry or anti-entropy reconciliation were stored but not forwarded.

What it actually was. The “first-seen forwarding” rule (forward exactly the first time, regardless of source) was implemented in only one of three acquisition call sites. The other two silently broke it.

How we fixed it. Centralised the `plumCancelGraftsFor(id) + reFanout(peer, id)` hook inside `fetchFrom` after a successful `Store.Put`. All three call sites inherit it.

Why the fix is right. The rule libp2p GossipSub applies: “not seen before then forward to mesh, irrespective of arrival method”. One place in the code, one behaviour.

Tag. v1.0.5, 20 April 2026.

5.6 v1.0.6: trust-aware fanout, parallel sends, dial-backoff

What broke. Cross-mesh publish latency regressed to about four minutes. Fanout was iterating over every peer serially; each dial to an unreachable peer paid Pilot’s full 32-second timeout; and the retry scheduler dialled the same dead peer again for every pending message.

What it actually was. Three independent gaps compounded. Fanout did not ask “is this peer even reachable” before attempting. Fanout was sequential so one slow peer blocked everyone. There was no backoff memory so the same dead peer was dialled over and over.

How we fixed it. Three coordinated changes:

- **Trust-aware reachability oracle.** Before building the fanout set, call Pilot’s `TrustedPeers` and filter. Peers with no trust pair (structurally unreachable for the life of this config) are dropped up-front. Cached for one second so bursty fanouts do not ask Pilot per message per peer.
- **Parallel fanout with per-peer timeout.** One goroutine per peer, each with a five-second `context.WithTimeout` and a concurrency cap of 32. One slow peer no longer blocks the others.
- **Per-peer dial-backoff cache.** After one failed dial, further dials short-circuit for an exponentially expanding window (one second, two, four, eight, capped at five minutes). A successful dial clears the window.

Together they collapse cross-mesh latency from roughly four minutes back to sub-second.

Why the fix is right. Each of the three changes is the canonical pattern from a mature system: the reachability oracle is how libp2p GossipSub treats connectedness; parallel fanout is idiomatic Go (`errgroup` with `SetLimit`); the dial-backoff cache is how `go-libp2p swarm.DialBackoff` works. Together they are the standard triptych for making “don’t spend real time on a dead peer” work.

Tag. v1.0.6, 20 April 2026.

5.7 v1.0.7: inline bodies and jittered backoff

What broke. Even after v1.0.6, a 1.5-minute propagation tail persisted. Each gossip hop paid two separate Pilot stream dials: sender’s push, receiver’s fetch-back for the body. And the deterministic retry schedule synchronised every node’s retries to the same wall-clock moments, so a single NAT flap cascaded into many minutes of propagation delay.

What it actually was. The push layer had shipped only the message ID, expecting the receiver to pull the body. The receiver’s pull was a whole new stream dial. And exact retry delays made independent nodes retry in lockstep.

How we fixed it. Two changes:

- **Inline body on eager push.** Gossip frames for messages smaller than 4 KiB now carry the full body. Receivers hash-verify and store directly, skipping the fetch-back round-trip. This is what the canonical Plumbtree paper specified in the first place; every surveyed production gossip (libp2p GossipSub, Scuttlebutt EBT, Matrix federation, Cassandra, IPFS Bitswap) does this.
- **Decorrelated-jitter retry backoff.** Replaces the fixed schedule with `next = min(cap, random(base, prev × 3))`, Marc Brooker’s 2015 AWS formula. Same 10-attempt budget, but nodes no longer retry in synchronised waves.

The reconcile cooldown also dropped from 60 seconds to a jittered 30 seconds (that range matches what GossipSub and Plumbtree §3.4 use for lazy-push cadence, and is an order of magnitude more appropriate than what had been in place).

Why the fix is right. Inline bodies bring Entmoot back to what the Plumbtree paper actually described; Entmoot’s earlier ID-only fanout was a non-canonical simplification. Decorrelated jitter is the specific, named cure for retry-synchronisation cascades, which is exactly what we were seeing.

Tag. v1.0.7, 21 April 2026.

5.8 v1.0.8: invites should not include the issuer

What broke. See the Pilot v1.9.0-jf.6 entry above: self-amplified packet storm on the VPS, 5,900 packets per second, 210% CPU.

What it actually was. Every invite minted up to that point listed the issuer as a bootstrap peer. When the issuer received its own invite through ambient membership channels, the Pilot layer happily established self-tunnels.

How we fixed it. Two defence-in-depth changes:

- `entmootd invite create` excludes the issuer's Node ID from `bootstrap_peers` at mint time.
- Pilot `v1.9.0-jf.6` adds the `ErrDialToSelf` sentinel so that any future invite that happens to include self cannot cause a storm.

Legacy invites still work: the `Gossiper.Join` path already has a self-skip filter at every bootstrap strategy. The new defences close the remaining holes.

Why the fix is right. Cassandra's gossipier maintains the invariant that `live_endpoints` excludes self by construction. This is the same invariant, applied at mint time. Defence in depth: fix the source, and also fix the callee so a future regression cannot amplify.

Tag. `v1.0.8`, 21 April 2026.

5.9 `v1.1.0`: one persistent session per peer

What broke. Both peers on a node were observed failing simultaneously at the Pilot stream-dial layer, even while the underlying tunnel reported `encrypted=yes auth=yes`. Each dropped dial cost Pilot's full roughly 32-second timeout before gossip could give up and try a retry; with two peers wedged in parallel, the accumulated timeouts dominated propagation latency.

What it actually was. Every gossip frame was opening a fresh Pilot stream, paying the full SYN handshake per frame. On a healthy tunnel that overhead is negligible, but whenever a stream SYN/ACK silently wedged (middlebox, NAT timer expiry, any of several causes we had chased in the Pilot layer), gossip was exposed to the 32-second Pilot dial budget on the new stream, even though the tunnel itself was fine.

How we fixed it. The Pilot transport adapter now maintains one long-lived `hashicorp/yamux` session per peer and multiplexes every gossip stream over it. The first contact still dials a Pilot stream; subsequent frames open a `yamux` stream on the cached session with no extra handshake (zero round-trip new-stream). `yamux`'s built-in keepalive (30-second interval, 10-second timeout) detects silently-wedged sessions and closes them; the next dial opens a fresh session automatically.

Wire compatibility: Entmoot's JSON frame format is unchanged; `yamux` simply multiplexes the same frames over the session. Mixed `v1.0.x` and `v1.1.0` peers do *not* interoperate (`v1.0.x` expects a raw stream per frame; `v1.1.0` expects a `yamux` session), so every node in a group must upgrade together.

Why the fix is right. Multiplexed long-lived sessions are how every mature stream-based overlay handles the per-frame overhead problem: Consul, Nomad, Vault, and `libp2p`'s `yamux` transport all use the same library. Pushing the keepalive down into the multiplexer means the gossip layer does not have to build its own liveness check for sessions, and silently-wedged transports are detected and recovered without any gossip-layer plumbing.

Tag. `v1.1.0`, 21 April 2026. Files: `src/pkg/entmoot/transport/pilot/pilot.go`, new test `pilot_yamux_test.go`, new dependency `github.com/hashicorp/yamux`.

5.10 v1.2.0: transport-endpoint advertisements over Entmoot gossip

What broke. A three-peer live test designed to exercise Pilot’s TCP-fallback transport stack (which would let Entmoot keep running when UDP is blocked) showed that the fallback code path was operationally inert — the Pilot daemon’s multi-transport dialer had a valid TCP path compiled in, but `peerTCP` was always empty at dial time, so the dialer always fell through to the beacon-relay path instead.

What it actually was. Our fork’s daemon dutifully advertises its TCP endpoint to the central registry at register time. The upstream stock registry at `34.71.57.205:9000` silently drops the `endpoints` field in the register frame, so peers’ registry lookups return only the UDP `real_addr`. The field-drop is an upstream bug we cannot patch; any attempt to route around it by standing up a fork-built registry would force a mesh-wide rebuild every time someone on the fork pushed a new tag.

How we fixed it. We distributed per-peer TCP endpoints *through Entmoot itself*. Each member signs a small `TransportAd` record naming its current endpoints and gossips it through the existing Plumtree fanout. Receivers verify the signature against the roster entry, store-or-replace under Last-Writer-Wins semantics keyed on `(group, author)`, and install the endpoints into Pilot’s `peerTCP` cache via a new `SetPeerEndpoints` IPC command (Pilot `v1.9.0-jf.7`). The record is bounded at one row per `(group, member)` regardless of publish rate, and three layers of spam defence run in cost order on receive: schema-and-size validation (≤ 1 KiB), publisher allowlist (sender *must* equal author), and per-(peer, topic) token-bucket rate limit. Only after all three pass does the Ed25519 signature verification run. A latent second bug in anti-entropy also surfaced during the test but was deferred to `v1.2.1`: `fetchPeerRange` used `since = latestLocalTimestamp` as its lower bound, so gaps older than the local node’s newest message were silently excluded from the range query.

Why the fix is right. Routing control-plane state through the data-plane gossip channel is a canonical 2020s federated-protocol move: Matrix uses signed state events over the same federation stream that carries message content; IPNS-over-pubsub distributes name pointers over libp2p pubsub rather than a separate DHT write path. The LWW + publisher-allowlist pattern is the structural spam bound that Matrix room ACLs and GossipSub `v1.1` peer scoring both converge on. Retaining one row per `(group, author)` keeps storage structurally bounded at $O(N)$ in the member count regardless of publish rate — the same invariant IPNS records enforce for distributed name pointers.

Tag. `v1.2.0`, 21 April 2026. Files: `src/pkg/entmoot/wire/types.go`, `src/pkg/entmoot/store/sqlite.go` (new `transport_ads` table), `src/pkg/entmoot/gossip/gossiper.go` (new `onTransportAd` handler), `src/pkg/entmoot/transport/pilot/pilot.go` (new `SetPeerEndpoints` call), Pilot `pkg/driver/driver`. + `pkg/daemon/ipc.go`.

5.11 v1.2.1: Range-Based Set Reconciliation, background anti-entropy, and tunnel-up trigger

What broke. A second live test surfaced the anti-entropy bug that `v1.2.0` had merely postponed. Two messages published from one peer (Phobos, Italian residential NAT) during a brief reachability outage never propagated to the public VPS, even after the VPS’s reconcile fired on an unrelated fresh message from the same peer. The `MerkleReq` short-circuit correctly reported roots mismatched; the subsequent fetch step still couldn’t bridge the gap.

What it actually was. Two orthogonal causes working together. The first: `fetchPeerRange(since = t)` used the local node’s newest-message timestamp as the range’s lower bound, so any gap with a timestamp *older* than the local newest was excluded from the query. The second: reconciliation fired only on inbound gossip events, not on a background timer, so a quiet partition with no organic publish traffic could leave a gap sitting indefinitely even after both sides reconnected.

How we fixed it. Three changes landed together. The reconcile algorithm moved from the timestamp-cursor fetch to Range-Based Set Reconciliation (RBSR). RBSR recursively splits the 32-byte message-identifier keyspace into sub-ranges, exchanges a 16-byte fingerprint per sub-range, and descends only where fingerprints disagree. Bandwidth is proportional to the symmetric difference, convergence is $O(\log N)$ rounds, and gaps anywhere in the timeline are found by construction. The Pilot-transport adapter gained a `SetOnTunnelUp(cb)` callback that fires `maybeReconcile` within milliseconds of every newly-established yamux session (outbound and inbound both). And a background `reconcilerLoop` goroutine ticks every $30\text{ s} \pm 20\%$ jitter, picks the least-recently-reconciled roster member, and silently skips when the peer’s cached Merkle root still matches local — an idle mesh pays zero dials per tick at steady state.

Why the fix is right. RBSR is the canonical 2020s set-diff algorithm; Willow, Earthstar, and iroh all adopt it, as does the Nostr Negentropy protocol (NIP-77). The 30s background ticker matches Serf’s `PushPullInterval` default for LAN gossip, chosen for exactly this “partition healed but no organic publish” scenario. Tunnel-up event triggers match Secure Scuttlebutt’s Epidemic Broadcast Trees on-connect handshake and Automerger sync’s have-heads-on-connect pattern; tying anti-entropy to connection events, not just timers, lets a flaky network converge as fast as its reconnects allow. Wire-compatibility with v1.2.0 peers is preserved: the legacy `MsgRangeReq` and `MsgRangeResp` handlers stay active, and a v1.2.1 initiator that encounters a v1.2.0 responder observes an EOF and falls back silently.

Live field verification (2026-04-22). Re-running the original bug pattern on the live three-peer mesh with all peers on v1.2.1: VPS ↔ laptop converged in three RBSR rounds with `missing = 0`. VPS ↔ Phobos, after Phobos cold-restarted its Pilot daemon to clear the pre-existing tunnel wedge, converged in three rounds with `missing = 13` (Phobos had accumulated thirteen messages across partition events); the VPS store grew from 95 to 108 messages, an exact match on the RBSR-reported difference. The `OnTunnelUp` callback fired 2.1s after the yamux session became usable. Zero `session ended early` frames were observed, confirming the wire-compat fallback stayed dormant once the whole mesh was on v1.2.1. After convergence, the `idle-mesh tick skip (roots match cache)` debug log fired on the background ticker, confirming the cached-root-match optimization kept the quiescent mesh silent.

Tag. v1.2.1, 22 April 2026. Files: new package `src/pkg/entmoot/reconcile/` (the RBSR state machine), `src/pkg/entmoot/wire/types.go` (new `MsgReconcile` opcode), `src/pkg/entmoot/gossip/gos` (rewritten `reconcileWith`, new `onReconcileReq` and `reconcilerLoop`), `src/pkg/entmoot/gossip/transport.` + `src/pkg/entmoot/transport/pilot/pilot.go` (`SetOnTunnelUp`), `src/pkg/entmoot/store/{store,memory.` (new `IterMessageIDsInIDRange`). No Pilot change.

6 The second shakedown: hide-IP, TURN, and port 1004

The first shakedown ended with normal-mode gossip converging. The second began when we asked a harder question: can the laptop hide its real address and still participate in the mesh? That moved the problem from ordinary NAT traversal to strict TURN-mediated operation. In normal Pilot mode a peer may try direct UDP, TCP, beacon relay, cached connections, and peer

TURN endpoints. In strict `-outbound-turn-only` mode the rule is different: every outbound peer packet must leave through the local node's own TURN allocation, and if that path cannot send, Pilot must fail closed rather than leak the real address.

6.1 Pilot jf.8–jf.11b: TURN and push rotation

What broke. The laptop could publish a TURN address, but the rest of the mesh kept learning stale relay ports after Pilot restarts or Cloudflare allocation rotations. Entmoot's first implementation polled `pilotctl info` every 30 seconds and republished when the address changed. That worked, but it was both slow and wasteful: the common case is no rotation.

What it actually was. TURN relay addresses are state, not configuration. They are assigned by the provider and can rotate under the daemon. Treating them like a static `-advertise-endpoint` value left peers pinned to dead allocations.

How we fixed it. Pilot jf.8 added TURN client support and credential providers. The jf.11a series added the privacy posture flags and `Info` fields that let Entmoot verify `-hide-ip`. Pilot jf.11b added IPC `Subscribe/Notify` and a `turn_endpoint` topic. Entmoot v1.4.0 added `-hide-ip` transport ads, and v1.5.0 replaced steady-state polling with a single subscription. Old Pilot daemons still fall back to the 30-second poller.

Why the fix is right. The application layer needs to publish current reachability, but it should not have to poll a local daemon forever. Push turns a rare state change into a rare IPC event. It also makes the privacy check explicit: Entmoot can suppress advertised addresses, but only Pilot can prevent outbound source-IP leaks.

6.2 Pilot jf.12–jf.15.7: rendezvous and bilateral TURN

What broke. Once all three machines ran TURN-aware builds, the next deadlock was subtler. VPS knew the laptop's new TURN allocation, but the laptop had not installed the reciprocal permission for the VPS allocation. Packets left one side and disappeared at the other side's TURN permission check. Because UDP send APIs report only local send success, both daemons could appear healthy while no bytes arrived.

What it actually was. TURN permissions are bilateral and allocation-local. Installing a peer's endpoint once is not enough: a local allocation rotation can erase the permission, and a peer rotation can make a cached permission stale. The push path itself depends on the data path, so it cannot be the only recovery mechanism.

How we fixed it. Pilot jf.14 introduced a small Pkarr-style rendezvous service for signed TURN endpoint records; jf.14.2 added refresh and stale-cache eviction; jf.15.3 installed reciprocal `CreatePermission` state when peer TURN endpoints are installed; jf.15.4 made the rendezvous lookup fire in strict TURN-only mode; jf.15.6 allowed recovery PILA to fire for TURN-delivered frames whose observed remote address is not a direct UDP endpoint; and jf.15.7 added periodic peer re-lookup so a data-path break does not prevent endpoint refresh. Entmoot's role in this phase was to keep publishing current transport ads, replay persisted ads at startup, and expose enough trace signal to separate discovery failure from transport failure.

Why the fix is right. Rendezvous is discovery, not routing. It gives each Pilot daemon the fresh endpoint material needed to install permissions and evict stale cached connections. The actual route decision remains in Pilot, where `-outbound-turn-only` can win deterministically over direct UDP, TCP, beacon relay, and peer-TURN preferences.

6.3 Entmoot v1.5.1–v1.5.10: stale streams and reconcile recovery

What broke. After Pilot peers finally reached `encrypted=yes auth=yes`, Entmoot still failed to converge. VPS could open or attempt streams to laptop port 1004, but reconcile sessions ended early, timed out, or retried through the same dead yamux/Pilot stream. Store counts diverged even though the underlying mesh was alive.

What it actually was. Several bugs stacked in the same symptom. The responder side of RBSR could learn that it was missing message IDs but fail to fetch the bodies. Short push contexts leaked into background reconcile attempts. Request/response deadlines started too early, before Pilot stream establishment completed. Stale connection IDs from the Pilot IPC layer looked like generic dial failures, which poisoned Entmoot’s backoff instead of evicting the cached session. Finally, closing or closing-soon Pilot streams could be accepted into yamux and reused until higher-level timeouts fired.

How we fixed it. Entmoot v1.5.1 made responder RBSR fetch symmetric. v1.5.2 through v1.5.4 split reconcile attempt lifetimes from push lifetimes, separated Pilot dial budget from wire I/O deadlines, and evicted stale sessions on response timeouts. v1.5.5 preserved early Pilot IPC close frames so a freshly registered connection observes EOF immediately. v1.5.6 fixed transport-ad self-install and forwarded snapshot validation. v1.5.8 retried stale Pilot stream IDs without poisoning dial-backoff. v1.5.9 introduced an explicit Pilot/yamux session lifecycle manager with generation ids, draining, active-stream counts, and a `-trace-gossip-transport` mode. v1.5.10 added `-trace-reconcile`, Pilot readiness waiting at startup, and transport-owned stream error classification.

Why the fix is right. Reconcile is a correctness path, not a best-effort push. It must treat “the cached stream died” as a local session repair event, not as proof that the peer is unreachable. The current split is the useful one: gossip owns retry policy and anti-entropy semantics; the Pilot adapter owns IPC/yamux-specific error classification and session lifecycle.

6.4 Pilot jf.15.8–jf.15.21: the stream layer becomes explicit

What broke. The last visible symptom was the famous `:1004` timeout. Small frames could pass, but large reconcile responses stalled, retransmitted until max retry, and reset. Other failures came from accepting half-open streams before the final ACK, from stale daemon-side stream ids, or from listener ports that survived an Entmoot restart.

What it actually was. TURN routing had become mostly correct; the remaining bugs were in Pilot’s virtual stream layer. The old implementation mixed SYN/SYN-ACK/ACK, port listener dispatch, IPC accept, retransmit state, `TIME_WAIT` cleanup, and send errors in one large path. That made symptoms look like Entmoot reconcile bugs even when the failed invariant was lower-level stream lifecycle.

How we fixed it. Pilot added `-trace-streams`, explicit stream transition/removal logs, listener `Unbind`, a passive-open rule that exposes accepted streams only after the final ACK, stale passive-handshake reaping, 1024-byte stream segments so UDP and TURN paths do not rely on IP fragmentation, centralized TURN route policy, and connection-scoped IPC close reporting for send failures. The latest Entmoot code consumes those signals through typed stream classification rather than brittle string matching.

Why the fix is right. A stream must not disappear without a reason log, and an accepted stream must already be usable. Once Pilot made those lifecycle rules explicit, Entmoot’s remaining job became ordinary anti-entropy: drop stale sessions, retry once on a fresh stream, and fetch the missing bodies. That is why the final deployment could converge all three stores after the Pilot transport layer went green.

6.5 Entmoot v1.5.11–v1.5.19: mobile service peer primitives and release observability

What broke. After the transport and reconcile paths were green, the next question was not another packet bug. It was product shape: an iOS app cannot be expected to run a full always-on `pilot-daemon` plus `entmootd join` process in the background. We needed the infrastructure for an always-on service peer that can help intermittent clients catch up and publish without taking custody of their signing keys.

What it actually was. The existing group protocol already stored and reconciled messages, but it lacked first-class service-peer interfaces. There was no durable per-client mailbox cursor, no HTTP sync surface, no local device registry, no request-signature helper, and no explicit release metadata command to inventory which binary a peer was actually running.

How we fixed it. Entmoot v1.5.11 introduced the signer abstraction, message-ingest events, and mobile service-peer building blocks. v1.5.12 moved publish success fully behind the durable local accept boundary so trust IPC stalls do not make a stored message look failed. v1.5.13 added durable SQLite mailbox cursors. v1.5.14 exposed those cursors through local `mailbox` CLI commands. v1.5.15 added the loopback-first ESP HTTP bridge. v1.5.16 added phone-signed publish forwarding. v1.5.17 added device-authenticated HTTP requests and a local device registry. v1.5.18 added operator CLI helpers for device onboarding and request signing. v1.5.19 added `entmootd version` so release verification no longer depends on guessing which binary a service manager launched.

Why the fix is right. The phone should hold the phone’s identity; the service peer should hold infrastructure. That mirrors the operational lesson from Lightning-style mobile systems without copying their protocol: keep the always-on node available, keep authoring keys off the server when desired, and make sync explicit and durable. The ESP path does not change Entmoot’s consensus state; it packages the existing store, roster validation, and gossip fanout into interfaces an intermittent client can safely use.

6.6 Entmoot v1.5.20–v1.5.25: executable ESP, multi-group runtime, raw Pilot streams, and hostname-aware members

What broke. Once the ESP surface became usable by an iOS client, two practical gaps appeared. First, mobile operations beyond message publish still required operator-side manual steps: group creation, group metadata updates, invite creation, and invite acceptance were queued as sign requests but not executed end to end. Second, live propagation still had stale-session failure modes because Entmoot kept a cached yamux session over Pilot streams even though Pilot itself already provides stream semantics.

What it actually was. The ESP needed a clear split between phone authorization and service-peer execution. The phone signs a canonical operation payload; the ESP verifies it and then asks the running `join` process to perform the operation. In parallel, the transport layer needed to stop layering a long-lived application multiplexer over a lower-layer virtual stream.

Each gossip or reconcile interaction is now a short-lived Pilot stream with explicit per-peer dial budgeting and write deadlines.

How we fixed it. v1.5.20 added the expanded ESP HTTP surface: session/status, groups, members, notification state, and durable sign requests. v1.5.21 made group and invite operations executable and added a bounded /history endpoint for initial mobile timeline rendering without moving mailbox cursors. v1.5.22 removed the yamux session cache above Pilot and moved to one Pilot stream per interaction. v1.5.23 and v1.5.24 restored tunnel-up callbacks and made fanout budgets Pilot-aware so TURN recovery and NAT refresh get enough time without blocking every message on a fixed long timeout. v1.5.25 forwarded those Pilot transport capabilities through the multi-group mux so every active group uses the same retry and stale-stream policy.

What came next. The current unreleased work adds the display metadata layer needed by mobile clients. Groups can carry ESP-local **name**, **description**, **tags**, and raw **metadata**. Members can publish signed **MemberProfileAd** frames containing their current Pilot hostname; peers store only roster-matched, unexpired profiles and new joiners pull a profile snapshot during bootstrap. This gives apps readable names such as **laptop**, **vps**, and **phobos** without changing Pilot registry behavior or Entmoot roster identity.

Why the fix is right. The pattern is the same in both places: keep authoritative protocol state small and signed, and put service/UI convenience in explicitly local or signed cache layers. ESP metadata does not rewrite rosters. Member hostnames do not decide routing. Raw Pilot streams remove a stale cache above the transport instead of adding another recovery loop around it.

7 The big picture: how the layers interacted

Many of the bugs documented above *looked like* problems in Entmoot but were actually problems in Pilot leaking upward. The four-minute propagation delay we chased on and off for two days is the best illustration.

Tracing it from the user's perspective back to the wire:

1. A user publishes on the VPS. The CLI returns instantly (v1.0.3), so the user knows the local store has the message, but they can see from the tail on the laptop that the message does not arrive.
2. The Entmoot gossip layer on the VPS runs a fanout. At that time the fanout was sequential and had no reachability filter, so it tried every peer serially.
3. Each attempt called Pilot's **DialConnection**. For the laptop peer, the dial landed in a timeout loop because (a) the cached UDP address was stale from NAT drift on the laptop's carrier, and (b) even when the tunnel was **encrypted=yes auth=yes**, the stream dial timed out because Pilot was reply-routing to a stale direct address instead of the relay path the last authenticated frame arrived on.
4. Pilot eventually returned a timeout after about 32 seconds. Entmoot queued a retry. The retry loop redialled the same dead peer after a fixed delay, synchronised with every other pending message, and paid another 32 seconds.

So the four-minute number was the product of one Pilot bug (NAT drift refresh, v1.9.0-jf.2; and reply on ingress, v1.9.0-jf.4), one Entmoot bug (serial fanout with no reachability filter, v1.0.6), and one Entmoot bug compounded by a second (deterministic retry schedule without

jitter, v1.0.7). Every patch peeled one layer off the onion. Only after all of them shipped did the latency number come back to sub-second.

The lesson for the architecture paper is that the split between layers should be clean enough that operators know where to look. Most of our time during this shakedown was spent deciding which layer owned a given symptom. After the dust settled the rule is simple:

- If `pilotctl peers` shows a peer as `encrypted=yes auth=yes` but `pilotctl ping` still times out, the problem is Pilot.
- If `pilotctl ping` succeeds but Entmoot fanout does not deliver, the problem is Entmoot.

v1.9.0-jf.4 and v1.0.6 landed in the same day because that is when we untangled which was which.

The second shakedown refined the rule. If Pilot peers are not encrypted and authenticated, the failure is still below Entmoot. If they are authenticated but port 1004 dials time out, first inspect Pilot stream traces: the bug may be in virtual-stream lifecycle even though it manifests as an Entmoot reconcile timeout. Only once `pilotctl ping` and stream traces are clean should the focus move to RBSR, store filtering, or gossip retry policy.

8 Cheat sheet

Two tables, one per layer.

Appendix A: reading order

If you only have time for three sections, read Section 2 (the mental model), Section 3 (the glossary), and Section 7 (how the layers interacted).

If you want to understand what broke on the wire, pick a single release from Section 4 or 5; each subsection is self-contained and uses the same four-part template (*what broke, what it actually was, how we fixed it, why the fix is right*).

The CHANGELOGs in `CHANGELOG.md` and `repos/pilotprotocol/CHANGELOG.md` are the source of truth for the technical claims in this document. The research paper (`paper/main.tex`) is the companion to this companion: the design as intended, with this document as the field report.

Version	Date	Summary
v1.7.2-jf.1	18 Apr	Skip NAT punch to private addresses.
v1.7.2-jf.2	18 Apr	Reset keepalive counter after rekey so tunnels stop flapping every 5 minutes.
v1.7.2-jf.3	19 Apr	Independent beacon keepalive (25 s); set-public persists to config.
v1.8.0-jf.1	19 Apr	Pluggable transport layer and TCP fallback.
v1.9.0-jf.1	19 Apr	Gossip peer-discovery overlay, decoupling reachability from the registry.
v1.9.0-jf.2	19 Apr	Refresh cached endpoint on every authenticated decrypt; macOS codesign workaround.
v1.9.0-jf.3	19 Apr	Auto-recovery PILA on crypto-state desync.
v1.9.0-jf.4	20 Apr	Reply-on-ingress path routing; 3 s SYN retransmit for relay mode.
v1.9.0-jf.5	20 Apr	Full per-peer state cleanup across AddPeer/RemovePeer .
v1.9.0-jf.6	21 Apr	ErrDialToSelf guard against self-tunnel storm.
v1.9.0-jf.7	21 Apr	New SetPeerEndpoints IPC command: Entmoot installs per-peer TCP endpoints into peerTCP without registry mediation.
v1.9.0-jf.8–jf.11b	24–25 Apr	TURN client support, hide-IP posture flags, no-registry endpoint mode, outbound-turn-only semantics, and IPC Subscribe/Notify for TURN rotation.
v1.9.0-jf.12–jf.14.2	25–26 Apr	Handshake reply routing, keepalives, rendezvous endpoint publish/refresh, and stale endpoint eviction.
v1.9.0-jf.15–jf.15.7	26 Apr	TURN allocation self-heal, permission refresh diagnostics, bilateral CreatePermission , recovery PILA through TURN, and periodic peer rendezvous re-lookup.
v1.9.0-jf.15.8–jf.15.13	26–27 Apr	Preserve TURN ingress, close stale IPC stream conns, recover fatal TURN allocation failures, centralize TURN route policy, and prefer own relay in strict TURN-only mode.
v1.9.0-jf.15.14–jf.15.21	27 Apr	Stream lifecycle tracing, listener Unbind , 1024-byte stream segments, passive-open ACK gating, stale handshake reaping, rendezvous 429 retry, and connection-scoped IPC send failures.

Table 1: Pilot fork release summary through the v1.9.0-jf.15 line.

Version	Date	Summary
v1.0.0	17 Apr	SQLite store, control-socket IPC, five-command CLI, invites with TTL.
v1.0.2	19 Apr	Exponential-backoff retry; anti-entropy reconciliation on reconnect.
v1.0.3	19 Apr	<code>entmootd publish</code> returns immediately after local store commit.
v1.0.4	20 Apr	Plumtree dissemination with IHAVE/GRAFT/PRUNE.
v1.0.5	20 Apr	Re-fanout fires on every acquisition path (push, retry, reconcile).
v1.0.6	20 Apr	Trust-aware fanout, parallel sends with per-peer timeout, dial-backoff cache.
v1.0.7	21 Apr	Inline message body on eager push; decorrelated-jitter retry backoff.
v1.0.8	21 Apr	Exclude issuer from <code>bootstrap_peers</code> at invite mint.
v1.1.0	21 Apr	Persistent yamux-multiplexed session per peer; no per-frame Pilot stream dial.
v1.2.0	21 Apr	Signed <code>TransportAd</code> records gossiped through Entmoot; Pilot <code>SetPeerEndpoints</code> IPC makes TCP fallback actually reachable.
v1.2.1	22 Apr	Range-Based Set Reconciliation; background AE ticker; <code>OnTunnelUp</code> callback; gap-reconcile bug fixed.
v1.3.0	25 Apr	Apache-2.0 licensing boundary and independent Pilot IPC client.
v1.4.0–v1.4.6	25 Apr	Hide-IP transport ads, TURN endpoint publication, cross-layer privacy warnings, TURN polling fallback, and IPC error routing hardening.
v1.5.0	25 Apr	Pilot <code>Subscribe/Notify</code> support; TURN rotation becomes push-driven with polling only as fallback.
v1.5.1–v1.5.4	26–27 Apr	Responder-side RBSR body fetch, stale-session recovery, separate dial/I/O deadlines, and post-timeout session eviction.
v1.5.5–v1.5.8	27 Apr	Preserve early Pilot IPC closes, ignore self transport ads, validate forwarded snapshots, and retry stale Pilot stream IDs without poisoning dial-backoff.
v1.5.9–v1.5.10	27 Apr	Explicit Pilot/yamux session lifecycle manager, transport trace mode, Pilot readiness wait, reconcile trace mode, and transport-owned stream error classification.
v1.5.11–v1.5.12	28 Apr	Mobile service-peer primitives, signer abstraction, ingest events, and publish success after durable local accept.
v1.5.13–v1.5.16	27–28 Apr	Durable mailbox cursors, local mailbox CLI, ESP HTTP bridge, and phone-signed publish forwarding.
v1.5.17–v1.5.19	28 Apr	ESP device-auth registry, request-signing helper, and <code>entmootd version</code> release metadata.
v1.5.20–v1.5.21	29 Apr	Expanded ESP mobile API, executable group/invite sign requests, ESP-local group metadata, and bounded latest-history reads.
v1.5.22–v1.5.25	29 Apr	Raw Pilot streams per interaction, stale stream invalidation, Pilot-aware fanout budgets, <code>tunnel-up</code> callback repair, and multi-group transport capability forwarding.
unreleased	30 Apr	Pilot hostname-aware member profiles plus ESP group <code>name/description/tags</code> display projection